

Thesis Setup — Full Explanation

Deploying YAGO locally with QLever, querying it from Python, and the metrics plan

Author: Dharmang Pambhar | Date: 9 June 2026 | Advisor: Samuel García (TUM)

This document explains, from the ground up, what the thesis task is, the tools involved, everything that was set up on the machine, how the pieces fit together, and what remains. It is meant to be readable on its own and useful to bring to an advisor meeting.

1. What this project is about

The thesis goal is to **compute metrics over a knowledge graph** (YAGO or DBpedia) and present them in a dashboard. A *knowledge graph* is a large set of facts written as **triples** — *subject – predicate – object*, e.g. Einstein – bornIn – Ulm. The pipeline has four layers:

```
KNOWLEDGE GRAPH DATA      YAGO / DBpedia (RDF triples)
  | loaded into
  v
QLever = the database / RDF store
  - 'index' once -> builds fast lookup structures on disk
  - 'start'      -> serves a SPARQL endpoint (a server on a port)
  | queried via SPARQL over HTTP
  v
YOUR METRICS CODE (Rust + SPARQL, like Knowgly)
  - SPARQL counts -> Rust HashMaps -> compute metrics
  | display
  v
niceGUI = the dashboard (Python -> charts, tables in the browser)
```

The four tools map cleanly onto these layers:

Tool	Role	Analogy
YAGO / DBpedia	The data (a knowledge graph of facts)	the raw material
QLever	The database that stores & queries it via SPARQL	like Postgres, but for graphs
Knowgly	A reference metrics project (to understand, not copy)	a worked example
niceGUI	The dashboard to display your metrics	the front-end

2. What QLever actually is

- **Triples & SPARQL**: RDF data is triples; **SPARQL** is its query language (like SQL is for tables).
- **QLever** is a very fast SPARQL engine from the University of Freiburg.
- **qllever index** reads the raw triple files *once* and builds an optimized index on disk (the slow, one-time step).
- **qllever start** launches a server that answers SPARQL queries over HTTP — so you can query it from Python.
- A **Qleverfile** is the recipe/config: which dataset, how to index it, which port, etc.
- The `qllever` command you install via pip is only the **controller**; the heavy engine runs separately (here, inside a container).

3. The machine and its constraints

Knowing the hardware explains several decisions made during setup.

Property	Value	Why it mattered
Computer	MacBook Air, Apple Silicon (arm64), 16 GB RAM	Limited RAM -> can't index 1.3B triples from scratch comfortably
Disk free	~140 GB at start	Enough for the 24 GB zip + 41.6 GB index
Docker	Not installed; no Homebrew; no admin sudo in tooling	Had to install a Docker engine without a password
Python	System Python 3.9 (too old)	qllever needs Python 3.12+

4. Everything that was set up (in order)

Step	What & why
1. uv + Python 3.12	Installed 'uv' (user-space Python manager) and created a Python 3.12 virtual environment, because the system Python 3.9 was too old for qllever.
2. qllever CLI	Installed the qllever control tool into the venv (uv pip install qllever).
3. Qleverfiles	Downloaded the YAGO, DBpedia and olympics recipes from the qllever-control repo.
4. Pre-built YAGO index	Downloaded the advisor's ready-made YAGO 4.5.0.2 index (skips the hours-long 'index' step).
5. Verified the download	First download was truncated by ~1 GB; proved it via Zip64 analysis and re-downloaded the full 23.88 GB, then extracted 41.6 GB.
6. Docker engine (Colima)	Installed Colima + Lima + Docker CLI into ~/.local/bin with NO admin password, and booted a lightweight Linux VM.
7. qllever start	Served the index at localhost:9004 using a QLever image pinned to the exact build that created the index.
8. Python querying	Wrote sparql.py + metrics_examples.py and confirmed real results against the local index.
9. Web UI	Launched the QLever UI at localhost:8176 and pointed it at the local index.
10. Moved off iCloud	Discovered the Desktop is iCloud-synced (it corrupted the venv); moved the whole project to ~/thesis (not synced).

5. The YAGO index (the data)

The advisor shared a finished QLever index, so the slow 'qllever index' step was skipped. It is **YAGO version 4.5.0.2**. Key facts read from its metadata:

Property	Value
Triples (normal)	1,305,431,407 (~1.3 billion)
Distinct subjects (entities)	49,688,174
Distinct objects	645,362,721
Distinct predicates	123
On-disk size	~41.6 GB (28 files)
Index build commit	e5f672 (Oct 2024)
Vocabulary	on-disk compressed dictionary (IRIs <-> integer IDs)

An index is made of **six permutations** (the triples sorted in every order: SPO, SOP, PSO, POS, OSP, OPS) so QLever can look up data quickly in any direction, plus the **vocabulary** (the dictionary that maps every IRI/string to an integer ID).

6. The query engine (Colima / Docker)

QLever's engine is a Linux C++ server with **no native macOS binary**, so it runs in a container. Docker was installed **without any admin password** using user-space binaries:

- **Colima + Lima + Docker CLI** placed in `~/local/bin` (no Homebrew, no sudo).
- Colima boots a small Linux VM using macOS's built-in Virtualization framework (vz).
- Your `~/thesis` folder is shared into the VM (virtiofs) so the container can read the 41.6 GB index.
- The QLever image is **pinned** to `commit-e5f6724` — the exact build that made the index — so the index format is guaranteed compatible ('latest' could reject a 2024-format index).

Endpoint: **http://localhost:9004** | Access token: `yago-4.5.0.2`

Note: visiting `http://localhost:9004` in a browser shows "Unknown path". That is normal — it is a SPARQL API, not a web page; it answers queries, not page requests.

7. The project files and folders

```
~/thesis/
  qlever-workspace/      <- project root
    rust_metrics/       <- THESIS METRICS (Rust + SPARQL, mirrors Knowgly)
      Cargo.toml
      src/qlever_client.rs <- SPARQL endpoint client
      src/common.rs       <- reusable queries
      src/entity_type_importance.rs <- the metric
      src/main.rs         <- runs it, prints results
    sparql.py            <- lightweight Python query reference (for dashboard)
    metrics_examples.py  <- Python example metrics (reference)
    make_pdf.py          <- generates this PDF
    RUNBOOK.md           <- start/stop/query/metrics commands
    Qleverfiles/         <- recipes (yago-4, dbpedia, olympics)
    yago/                <- THE DATABASE (41.6 GB index, not code)
    .venv/               <- isolated Python 3.12 + qlever + requests
    Thesis_Setup_Explained.pdf <- this document
```

Categories: **thesis metrics** (the Rust project `rust_metrics/`), **config** (`Qleverfiles`), **data + tooling** (`yago/`, `.venv/` — set up once), and a small **Python reference** (`sparql.py`) that will later feed the dashboard.

8. The metric implementation (Rust + SPARQL)

Per the advisor and the Knowgly reference, the metrics are implemented as a **mix of SPARQL and Rust working with dictionaries (HashMaps)**: SPARQL does the heavy counting on the server, and Rust holds the aggregated counts in nested HashMaps and applies the formula. The project is `rust_metrics/` and mirrors Knowgly's structure.

File	Role
<code>src/qlever_client.rs</code>	One global SPARQL endpoint; <code>query()</code> sends a SELECT and returns rows as <code>var->value</code> maps (reads results via a streaming reader so large result sets work).
<code>src/common.rs</code>	Reusable queries: <code>fetch_top_type_iris(N)</code> and <code>entity_count_per_type()</code> (<code> E_t </code>).
<code>src/entity_type_importance.rs</code>	The metric. Per-type SPARQL GROUP BY queries run in parallel (rayon); results stored in <code>HashMap<Type, HashMap<Predicate, score>></code> .
<code>src/main.rs</code>	Initialises the endpoint, runs the metric, prints each type's most-important predicates.

The implemented metric — Entity Type Importance (a TF-IDF for predicates):

$$ETImp(p, t) = EF_p(p, t) * \log_2(|E_t| / EF_p(p, t))$$

```
EF_p(p,t) = # distinct entities of type t that use predicate p    (SPARQL COUNT DISTINCT)
|E_t|      = # distinct entities of type t
```

Run it against your local index:

```
source "$HOME/.cargo/env"
cd ~/thesis/qllever-workspace/rust_metrics
export QLEVER_ENDPOINT=http://localhost:9004
cargo run --release 8          # top 8 most-populated types
```

Example output (characteristic predicates surface correctly): **Person** -> deathDate, birthDate, children, spouse; **Galaxy** -> radialVelocity, distanceFromEarth, parallax; **Politician** -> memberOf, birthPlace, deathPlace.

Note on scope: Knowgly computes this for *all* types; on a 16 GB laptop one query per type over 1.3B triples is slow, so we restrict to the top-N most-populated types (the `cargo run --release N` argument). Same metric, tractable scope.

Python files (reference only). `sparql.py` and `metrics_examples.py` are a small Python query helper kept for quick checks and to feed the future niceGUI dashboard — they are *not* the thesis metric implementation (that is the Rust project above).

9. How to start / stop / query (runbook)

Set up the shell each session:

```
export PATH="$HOME/.local/bin:$PATH"
source ~/thesis/qllever-workspace/.venv/bin/activate
```

Start everything (e.g. after a reboot):

```
colima start
cd ~/thesis/qllever-workspace/yago
qllever --qlleverfile Qleverfile start    # serves localhost:9004
```

Check / stop:

```
docker ps          # see the running server
qllever --qlleverfile Qleverfile stop    # stop QLever
colima stop        # stop the Docker VM (frees RAM)
```

Query from the browser UI: **<http://localhost:8176/yago-4>**

10. The Knowgly metrics — the idea

The advisor asked you to **understand** Knowgly's metrics (not reimplement them). The whole approach is one pattern:

Metric = SPARQL does the counting (GROUP BY ... COUNT) -> code plugs the counts into a formula -> results stored in nested dictionaries (Type -> Predicate -> value). The "Rust for dictionaries" the advisor mentioned is just bookkeeping of those counts; in Python it is a plain dict.

Knowgly implements two metrics that rank how important each predicate is for each entity type:

- **Entity Type Importance** (a TF-IDF for predicates): $ETImp(p,t) = EF_p(p,t) \times \log_2(|E_t| / EF_p(p,t))$, where $EF_p(p,t)$ is the number of entities of type t that use predicate p , and $|E_t|$ is the total entities of type t . Predicates used by every entity of a type score 0 (uninformative); characteristic predicates score high.
- **Entropy Type Importance** (Shannon entropy of a predicate's object values): $H = -\sum P(o) \log P(o)$. High entropy = diverse, informative values (name, birthDate); low entropy = near-constant values.

Engineering lessons worth reusing even for simple metrics:

- Do NOT write one giant query; split into many small per-type queries (avoids query-planner blowups).
- Run them in parallel.
- Aggregate the counts in dictionaries.
- Optionally cluster predicates by their scores into importance groups.

Your thesis metrics are **simpler than Knowgly's** but implemented the **same way — SPARQL + Rust dictionaries**. The first one, Entity Type Importance, is already built in `rust_metrics/` (see Section 8) and runs against your local YAGO.

11. Does this cover the supervisor's instructions?

Supervisor's instruction	Status
Explore / use QLever to deploy YAGO locally	DONE
Install qlever via pip	DONE (in a Python 3.12 venv)
Grab a Qleverfile	DONE (yago / dbpedia / olympics)
qlever ... index	SKIPPED ON PURPOSE - used the pre-built index instead
Use the pre-built YAGO index (SharePoint)	DONE (downloaded, verified, extracted)
qlever ... start	DONE (serving localhost:9004)
Query as a SPARQL endpoint with Python	DONE (sparql.py + metrics_examples.py)

Everything in the message is implemented. The only line not literally executed is `qlever index` — intentionally, because the advisor's pre-built index exists precisely so you can skip that slow step.

12. Current status and next steps

Thesis piece	Status
Deploy YAGO locally with QLever	DONE
Query SPARQL (Python + Rust)	DONE
Understand the Knowgly metrics	DONE
First metric in Rust + SPARQL (Entity Type Importance)	DONE

Add more metrics (e.g. entropy-based)	TODO
Build the niceGUI dashboard	TODO

The architecture is settled and the first metric works. What remains: add a few more metrics in the same Rust + SPARQL style, have the Rust program write its results (e.g. as JSON), and build a niceGUI dashboard that reads those results and charts them.